ACADEMIC PROJECT TECHNICAL REPORT & VIVA NOTES

NEXSOCIETY: SMART AUTONOMOUS GATED COMMUNITY PLATFORM

Full-Stack System Architecture, Codebase Breakdown & Backend Engineering Guide

Domain: Web Engineering & IoT / Cloud	**Architecture:** 3-Tier Multi-Role Full Stack	**Frontend:** HTML5, CSS3, JS, Three.js	**Backend:** Node.js, Express, PostgreSQL

1. Executive Overview & Problem Statement

Traditional residential societies face security loopholes (paper visitor registers, manual phone calls for guest verification, unrecorded deliveries), delayed maintenance fee collection, and zero visual oversight of campus assets.

NexSociety solves this by interconnecting three key society stakeholders onto a single unified real-time cloud platform:

Security Guards: Instant 1-tap fast pass for delivery partners (Zomato/Swiggy), visitor registration, and OTP-driven boom barrier automation.
Society Residents: Pre-approve guests with 6-digit entry PINs, raise helpdesk repair tickets, track maintenance dues, and access Gate 1 intercom.
Executive Secretary / Admin: Real-time oversight of flat occupancy, maintenance collection audit, directory search, and broadcast circulars.
3D Digital Twin HUD: Real-time WebGL interactive 3D model of Towers A, B, and C with live occupancy telemetry and parking bay status.

2. Complete System Architecture & Data Flow

> **Tri-Tier Resilience Architecture:** The application connects to a Node.js/Express REST backend with PostgreSQL. If the backend server is offline or experiencing network drops during presentation, an automatic client-side fallback routes data to **Firebase Realtime Database** and **LocalStorage** without any crashes.

```
[CLIENT APPLICATION] (Desktop / Mobile Devices)
       |
       ├──> [NETLIFY EDGE CDN] ──> Static Delivery: HTML5, CSS3, ES6 JS, Three.js
       |
       ├──> [REST API BACKEND] ──> Node.js / Express Server (Port 5000)
       |           |
       |           └──> [POSTGRESQL DATABASE] (Relational Tables: users, visitors, dues, complaint
       |
       └──> [FIREBASE CLOUD] ──> Realtime Database (Session Tracking: nexsociety-b4311)
```

3. Detailed File-by-File Breakdown

File Name	Core Responsibilities	Technical Implementation
`index.html` `script.js` `style.css`	Unified Authentication Portal & 1-Click Demo Launcher.	Glassmorphic CSS grid layout. `script.js` verifies role credentials (security, resident, admin), logs active session to Firebase RTDB, and stores user in `sessionStorage`.
`security.html` `security.js` `security.css`	Gate Guard Terminal (Gate 1 North). 1-Tap delivery check-in, live log, OTP boom barrier.	Dynamic DOM rendering for visitor tables, modal dialogs for incident reporting, 6-digit PIN verification, CSV export engine via client-side Blob generation.
`resident.html` `resident.js` `resident.css`	Resident Self-Service Dashboard (Flat A-204). Guest pass, dues invoice, complaints.	6-digit numeric pass generation, Web Audio simulated guard intercom with animated waveform, helpdesk filter tabs, dynamic circular notice banner.
`admin.html` `admin.js` `admin.css`	Executive Command Center. 120-unit society flats directory, maintenance collections.	Real-time KPI calculations (occupied vs vacant units), directory search filter, complaint resolution workflow, notice publishing form.
`payment.html` `payment.js` `payment.css`	Fintech Payment Gateway Module. Society maintenance fee checkout simulator.	Interactive 3D Credit/Debit card with CSS 3D flip animation, UPI apps deep link simulation, GST-compliant printable receipt generator.
`twin.html` `twin.js` `twin.css`	3D Digital Twin Command HUD. Interactive 3D architectural rendering.	Built with **Three.js** WebGL. Perspective camera, ambient and directional sunlight, procedural window shaders, and `THREE.Raycaster` for click telemetry.
`firebase-config.js`	Cross-device Firebase Cloud Connector.	Firebase v9 Compat SDK connector. Pushes user login metadata (email, role, flat, timestamp, user agent) to `/sessions` on Firebase Realtime Database.
`api.js`	Centralized HTTP API Request Client.	Unified Fetch API wrapper with timeout controller, Authorization headers, and seamless fallback to LocalStorage if backend is unreachable.

4. Backend Engineering (Node.js + Express + PostgreSQL)

The backend directory `server/` handles server-side business logic, security middleware, and database operations.

A. Server Configuration (`server/server.js`)

Port: Runs on `http://localhost:5000`.
CORS & PNA Headers: Configured with `Access-Control-Allow-Origin: *` and `Access-Control-Allow-Private-Network: true`, allowing frontend instances hosted on Netlify to query the local Express server.
Health Endpoint: `GET /api/health` checks PostgreSQL connectivity and returns database status.

B. PostgreSQL Database Schema (`server/db/schema.sql`)

Table Name	Primary Columns	Description
`users`	`id, full_name, email, password_hash, role, flat_number, tower`	Stores registered security guards, residents, and admins with role-based permissions.
`visitors`	`id, name, phone, flat_number, purpose, vehicle_no, entry_time, exit_time, status`	Gate security logs tracking active visitors inside campus and departed visitors.
`preapproved_passes`	`id, guest_name, guest_phone, flat_number, pin_otp, arrival_date, status`	Stores 6-digit numeric passcodes generated by residents for guest pre-authorization.
`complaints`	`id, flat_number, resident_name, category, priority, details, status`	Helpdesk maintenance requests (Plumbing, Electrical, Elevator) with review status.
`maintenance_dues`	`id, flat_number, resident_name, tower, amount, month, status, transaction_id`	Monthly dues billing tracking paid vs pending collections with transaction references.
`notices`	`id, headline, category, content, published_by, created_at`	Broadcast circulars published by society executive desk to all resident dashboards.

5. Core Feature Workflows (How It Works)

Workflow 1: 6-Digit Gate Pass Verification

Generation: In Resident Portal (`resident.js`), resident inputs guest name and flat. A random 6-digit numeric PIN is generated (e.g. `482910`) and saved to `preapproved_passes` .
Arrival: Guest arrives at Gate 1. Guard clicks *"Verify Passcode"* on the Security Terminal (`security.js`) and inputs `482910` .
Verification: The system queries the database. If valid, the pass status updates from `active` to `used` , boom barrier opens, and the visitor is automatically added to the live campus log with status `inside` .

Workflow 2: Fintech Maintenance Payment & Receipt

Resident views pending maintenance (₹4,250) on `resident.html` and clicks *"Pay Now"*, redirecting to `payment.html` .
Resident chooses payment mode: 1-Tap UPI (GPay/PhonePe) or Credit Card.
Submitting payment triggers `processPayment()` in `payment.js` . It validates card/VPA inputs, simulates a 2-second bank authorization gateway, generates a unique transaction reference (e.g., `TXN-NEX-88219`), marks dues as `PAID` , and generates a GST-compliant tax receipt.

Workflow 3: 3D Digital Twin Visualizer (Three.js)

`twin.js` initializes a WebGL scene with shadow mapping and perspective camera.
Renders 3 towers: Tower A (12 floors), Tower B (10 floors), Tower C (8 floors) with distinct textures.
Adds campus elements: boundary walls, boom barriers, parking bay meshes, and landscaping.
A `THREE.Raycaster` listens to user pointer clicks. When a user clicks Tower A, it calculates intersection points, highlights the building, and slides open the telemetry HUD showing occupancy (96/120 flats occupied).

6. Frequently Asked Viva Questions & Model Answers

Q1: What is the overall architecture of NexSociety?

Answer: "NexSociety follows a modular 3-tier client-server architecture. The frontend is built using semantic HTML5, CSS3, ES6 JavaScript, and Three.js for 3D visualization. The backend is an asynchronous REST API built with Node.js and Express.js, connected to a PostgreSQL relational database. For cloud deployment and real-time cross-device monitoring, we use Firebase Realtime Database and Netlify Edge CDN."

Q2: Why did you choose PostgreSQL over MongoDB?

Answer: "Residential society management involves financial transactions (maintenance dues) and structured relational associations (visitors linked to flat numbers, complaints linked to towers). PostgreSQL guarantees ACID compliance, strict foreign key constraints, and numeric precision for financial ledgers, preventing dirty reads or lost updates."

Q3: How does your application handle network interruptions during a demo?

Answer: "We implemented a dual-mode offline-resilient architecture in `api.js` and `script.js`. If the local Express backend (port 5000) is unreachable, the frontend seamlessly falls back to LocalStorage and Firebase Realtime Database. The UI never crashes or displays broken errors."

Q4: How did you ensure responsive mobile screen fitting without horizontal scroll?

Answer: "We applied strict global viewport bounds: `html, body { width: 100%; max-width: 100vw; overflow-x: hidden; }`. Wide multi-column tables are wrapped in `.table-responsive` containers with `overflow-x: auto` and touch momentum scrolling. Media queries at 768px and 480px dynamically convert multi-column grids into single-column responsive layouts."

Q5: How does the 3D Digital Twin work under the hood?

Answer: "The 3D Digital Twin uses the Three.js library over WebGL. It sets up a scene, camera, ambient and directional lights, and renders procedural 3D box geometries for towers and parking lots. User click interactions are detected using `THREE.Raycaster`, which projects a ray from the camera through the mouse coordinates to identify clicked building meshes."